

单词

给定一篇文章的所有组成单词，问文章中的每一个单词在文章中出现了多少次。

输入：

3

a

aa

aaa

输出：

6

3

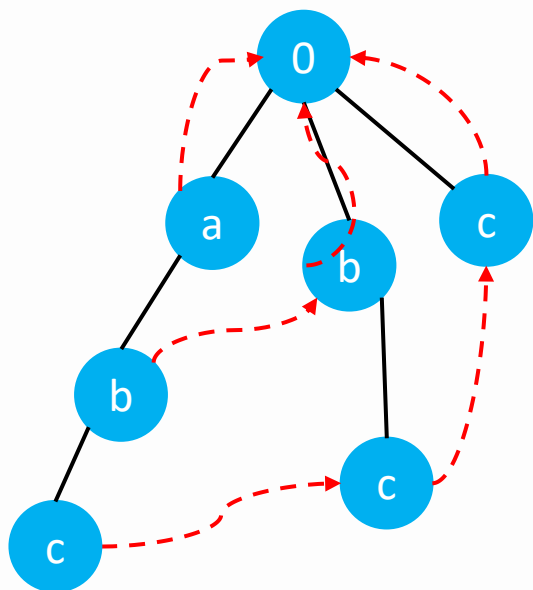
1

原文为 a aa aaa, 那么单词a出现了6次，单词aa出现了三次(在aaa中算出现了两次)，aaa出现了一次。

单词

该题并不是问这个字符串是否出现过，而是问这个字符串是否是其中哪些字符串的子串(只要该字符串被包含在内都算)。而且该题是反过来的，不是问你字典树上哪些字符串是该字符串的子串，而是问该字符串是字典树上哪些字符串的子串。

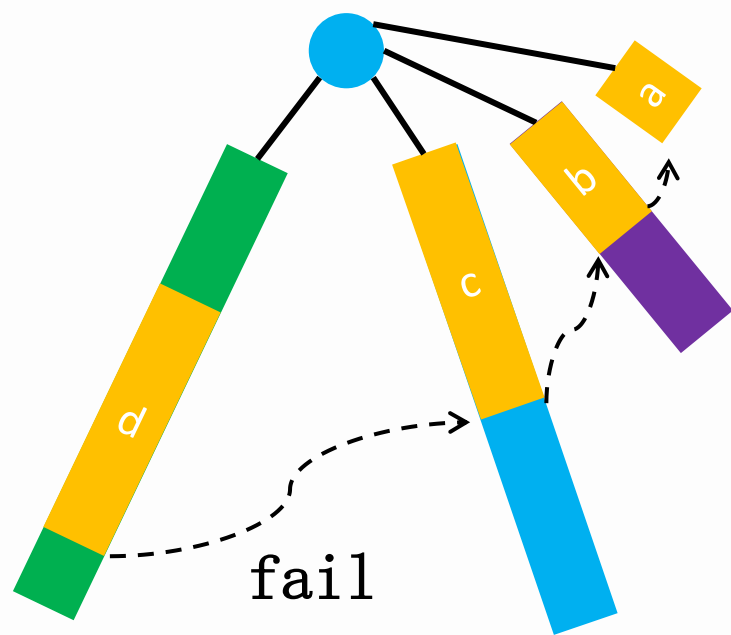
假设有三个单词abc, dc, c



我们要查找单词c出现的次数，应该是3次，但是如果我们去AC自动机中去查找c，只能找到一个。

那我们应该如何处理呢？

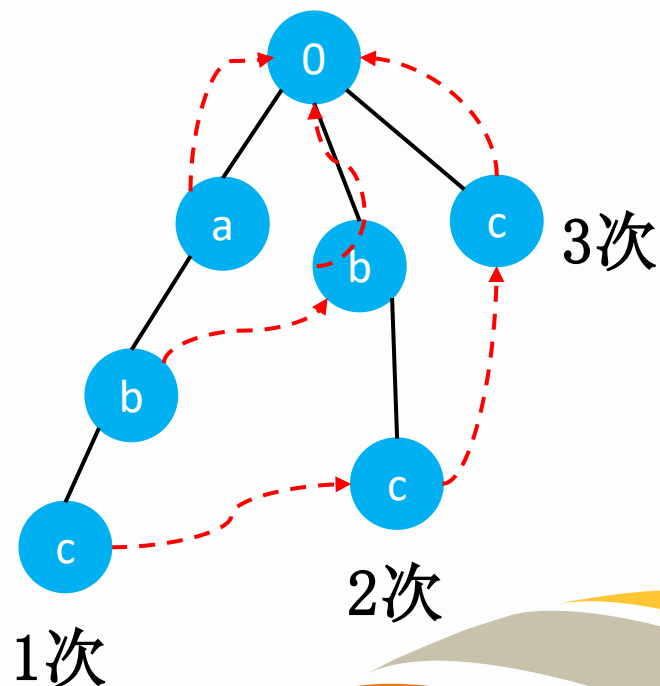
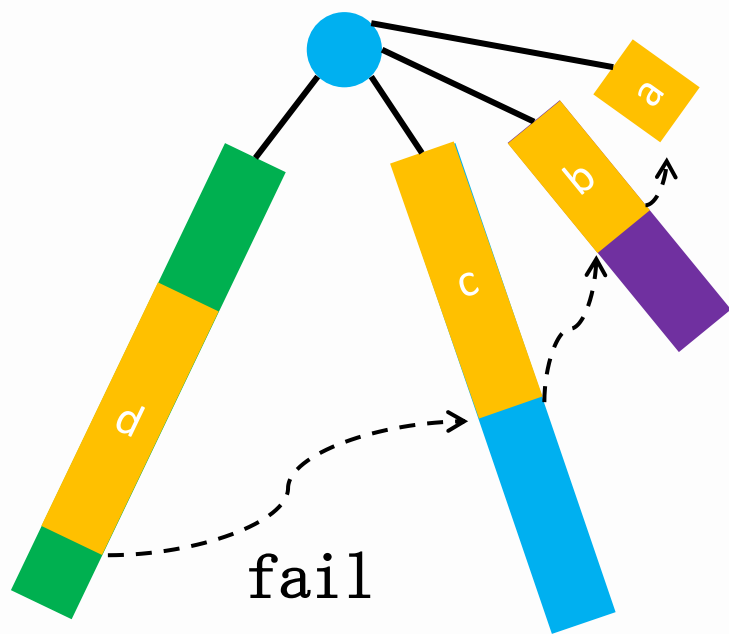
单词



假设我们要查找黄色部分a这个单词出现的次数，那么如果有一个单词的fail指针指向了a这个单词的末尾，那么就说明a单词在b单词中出现过。出现次数+1，我们继续往下面看，如果有一个单词c的fail指针指向了b，那么说明单词b存在于单词c中，所以单词a也存在单词c中，出现次数+1. 我们可以一直这样找下去，直到找不到为止，中途没fail指针跳一次，那么出现次数就会+1次。

单词

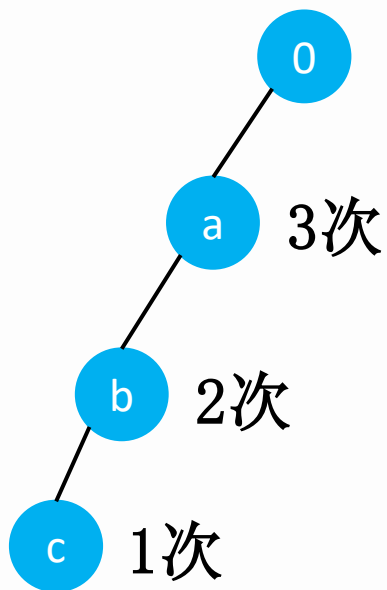
那么我们就可以开一个sum数组，表示有多少个包含的单词，那么 $\text{sum}[\text{fail}[i]] += \text{sum}[i]$



单词

第一个问题解决了，但是这个sum的值该如何赋值呢？
每添加一个节点，那么sum就+1.

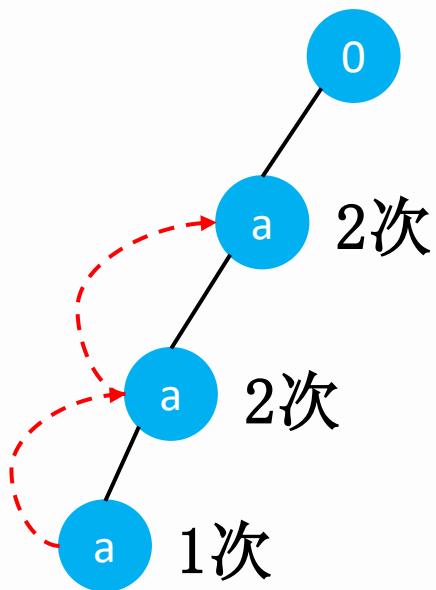
比如插入abc, ab, a。



单词

但是这样做有一个很大的问题，也就是说如果一个单词中如果一个字符或者一截字符串重复了多次，这样统计答案就错了。

比如aaa,, 单词a出现的次数应该是3次，但是这里明显只有2次。 $\text{sum}[1]=\text{sum}[2]+\text{sum}[1]=2$
 $\text{sum}[2]=\text{sum}[3]+\text{sum}[2]=2$



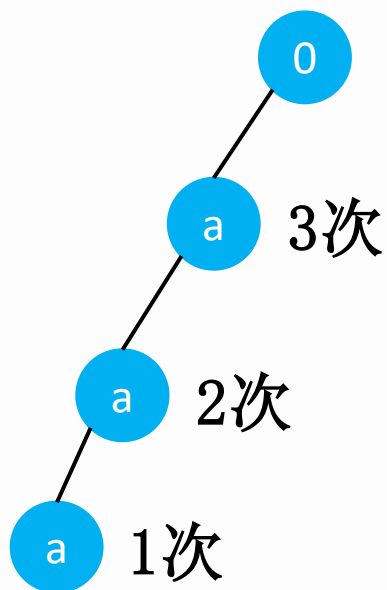
所以我们统计sum数组的时候应该在fail指针全部建完之后再单独统计，不能边建边统计。

同时统计的时候应该从链的底端开始往上面统计。

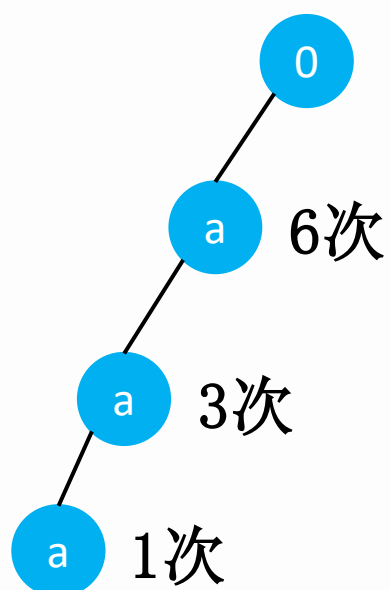
单词

比如aaa, aa, a, 我们来尝试一下

先全部插入



再统计



hash

Hash是一个神奇的算法，他主要用于字符串的比较，但是又不仅仅用于字符串的匹配，他主要的作用是把一串比较繁琐的字符转换成与之对应的简单数值（哈希值），这样我们每次比较的时候，只需要比较对应的数值就可以了。我们知道字符串的比较是 $O(m)$ 的，但是数字的比较却是 $O(1)$ 的，所以在效率上提升了不少。

既然原字符串和哈希值是一一对应的，那么转换就必须遵循一个原则：对于同一个字符串，哈希值必须是相同的，同样的，对于不同的字符串，哈希值必须是不相同的。否则就没有办法通过判断哈希值来直接判断两个字符串是否相等。所以，设计一个优秀的hash函数是特别重要的。

hash

根据进制数的原理，我们知道任何一个进制数都没有重复的，比如对于任意一个x进制的数，他的每一位都小于x，实际上这个数表示的十进制数就是

$$s_0 * x^i + s_1 * x^{(i-1)} + \dots + s_i * x^0$$

那么我们也可以把字符串这样转换成一个数字，并且这样转换的数字是没有重复的。

因为字符都是小于128的，所以我们可以给每一个字符乘以一个大于128的数把他转换。

比如abc, 我们可以转换成

$$'a' * x^2 + 'b' * x^1 + c$$

这样转换了之后得到的数字肯定是没有重复的

比如这个数字我们可以设置成131(质数)，为什么要质数，我们接下来再讲。

hash

但是这样做也有一个很大的弊端，那么就是数字溢出，因为每一次乘以131，那么字符串长度稍微长一点就会溢出，如果用高精度存储，那么也就相当于原字符串，没有任何改进，所以为了保证不溢出，所以我们需要取余。

但是一旦取余，那么就会出现不同字符转换成的数字相等了，就和我们的转换规则冲突了。所以我们要尽量避免冲突。

通过多年研究，一般会用到乘以一个大的质数，再除以一个大的质数的办法。这样转换得到的答案冲突最小。

甚至还有可以双hash来让冲突达到更小。

Hash函数

Hash函数的设计方法有很多种，网上也有很多dalao给出了数据测试以及比较，我们这里主要讲解一个乘以一个质数再对一个大质数取模的办法。

比如把一个字符串str转换成hash值：

```
for(int i=0;i<l;i++)  
{  
    sum=(sum*131+str[i])%1000007;  
}  
if(!book[sum])  
{  
    book[sum]=1;  
}
```

一般乘数取131，
除数取 10^n+7 ，
根据题目需要。

Hash函数

很多同学也想到了，这样做可能是有冲突的，也就是说就算这样做了，还是有可能不同的字符串转换成相同的数字，特别是取余之后，这就会导致答案错误。

的确，因为这样写连洛谷P3370 字符串哈希的裸题都过不了，更不要说在考场上使用了。

所以我们要做的就是降低冲突，甚至消除冲突，目前比较完美(既不降低效率也能百分百正确)的算法还没有，这里介绍一种不会冲突，但是效率会低一点的算法。

前面我们可能想到了一种解决办法，也就是说如果两个字符串的hash值不相等，那么说明这两个字符串一定不等，但是两个字符型的hash值相等，那么并不能说明这两个字符串一定相等，那对于后一种情况比较一下不就ok了

Hash算法

对于前面hash值相同的情况，我们可以比较一下两个原字符串是否相等，因为有人已经证明过了，冲突不会太多，所以不用担心会比较太多。但是比较也要注意，可能有很多个字符串的hash值都是一个数 k ，那么这些字符串你是都需要保存下来的，而不是只保存一个，所以我们采用数组挂链的形式。

即如果当前位置被占用了，并且这两个字符串通过比较之后发现并不相同，我们就存在该位置的下一个位置，但是很有可能下一个位置也被用过或者存的也是相同hash值的不同字符串，那么我们就接着往下找，直到找到空位置或者两个字符串相等为止。

这样就能让我们的算法肯定正确，但是很明显效率会低很多。

Hash算法

```
void hash(char *s, int k)
{
    int l=strlen(s);
    int sum=0;
    for(int i=0;i<l;i++)
        sum=(sum*131+s[i])%mod;
    while(book[sum]&&strcmp(str[book[sum]], str[k]))
        sum++; //一直向下一个空位找
    if(!book[sum])
    {
        ans++;
        book[sum]=k; //标记该位置是哪一个字符串
    }
}
```

注意下mod的值尽量大一点，这样数组中间的空位就会比较多，这样就算冲突也可以马上找到下一个位置，这就是典型的用空间换时间。



Hash算法

Hash不但能很快的解决判断字符串相等的问题，还可以很快的用来做字符串的匹配，也就是很多kmp能做的hash也能做，不过在保证正确性的前提下hash会比kmp要慢一点。

先思考一下如何用hash来做字符串匹配





Hash算法

- (1) 先得到匹配串的长度 l
 - (2) 在目标串中 $O(m)$ 的预处理出来所有长度为 l 的子串的hash值
 - (3) 再按照前面的做法来匹配。
- 

POJ 3461

题目大意：已知两个字符串s1, s2，判断字符串s1在s2中出现的次数

输入：

3

BAPC

BAPC

AZA

AZAZAZA

VERDI

AVERDXIVYER

DIAN

输出：

1

3

0

$L1 \leq 10000;$

$L2 \leq 1000000;$

POJ 3461

当然，该题是一道KMP的裸题，但是有些学生就是学不会kmp，就算学会了下次也忘了，那么我们可以考虑字符串hash。效率也差不多。但是具体的实现和我们前面有点不太一样。

(1) 一般不考虑冲突问题，由于考虑冲突问题效率比较低，所以可以直接认为hash值相同，字符串就相同

(2) 不取模，取模之后冲突的概率就会增加，所以干脆不取模，考虑自然溢出，即直接开long long，超出范围自然溢出，还是可以比较。

POJ 3461

(3) 由于我们对于每一个位置并不是直接相加，而是乘以一个质数再相加，那么实际上前面的数已经被乘了很多次了，所以我们要消除前面的不在区间内的字符的影响。



比如以第一个字符为开始字符长度为 l 的hash值为 $sum[1]$ ，那么以第二个字符开始的。。。值 $sum[2]$ ，他们之间关系是：

$$sum[2] = (sum[1] - s[1] * 131^{(l-1)}) * 131 + s[l+1]$$

POJ 3461

按照前面的计算方法有点麻烦，所以我们可以考虑前缀和。

如果我们要求区间 $[i, j]$ 的真实hash值，我们可以写成 $\text{sum}[j] - \text{sum}[i-1] * \text{pow}[l]$

乘方我们也可以先预处理出来。这些就比较方便

```
for(int i=1;i<=1000010;i++)
    pow[i]=pow[i-1]*131; //自然溢出

for(int i=1;i<=l2-l1+1;i++)
{
    if(sum==sum2[i+l1-1]-sum2[i-1]*pow[l1])
        ans++;
}
```

Friend

有三个好朋友喜欢在一起玩游戏, A君写下一个字符串S, B君将其复制一遍得到T, C君在T的任意位置(包括首尾)插入一个字符得到U. 现在你得到了U, 请你找出S. 如果串S不唯一, 输出NOT UNIQUE

输入:

7

ABXCABC

输出:

ABC

输入:

9

ABABABABA

输出:

NOT UNIQUE

$N \leq 2000000$

Friend

这道题看上去很简单，判断是否由一个字符串*2，中间再插入一个字符组成的，就纯暴力就可以做。插入的位置有五种情况，最前面，前一个字符串中间，两个字符串之间，后一个字符串中，最后面。

但是这道题关键是还需要判断情况是否唯一，比如ABABA，他的原字符串可以是AB，也可以是BA，都符合条件，还有一些更长的，他的情况或者新插入的字符的位置可能有很多种，所以我们需要枚举插入的位置，再判断是否符合条件，为了提交时间效率，我们要用到字符串hash。

Friend

我们先枚举插入字符的位置为 i ，原始字符串长度为 l ，总长度为 n 。有三种情况(五种情况可以简化为三种)。第一种： $i \leq l$ ，第二种 $i = l + 1$ ，第三种 $i > l + 1$ 。

(1) $i \leq l$



T1

T2

T4

我们只需要判断前后两部分的hash值就可以了，因为枚举的时间很大，所以我们计算hash值的时候需要 $O(1)$ 的计算，所以我们先预处理出来整个字符串的hash值，再计算部分的hash值。

$T1 = \text{hash}[i-1];$

$T2 = \text{hash}[l+1] - \text{hash}[i] * \text{pow}[l+1-i];$

$T3(\text{前一个字符串真实hash值}) = T1 * \text{pow}[l+1-i] + T2;$

$T4(\text{后一个真实hash值}) = \text{hash}[n] - \text{hash}[l+1] * \text{pow}[l]$

Friend

(2) $i == l+1$ 

T1

T2

$T1 = \text{hash}[l];$

$T2 = \text{hash}[n] - \text{hash}[l+1] * \text{pow}[l];$

(3) $i > l+1$ 

T1

T2

T3

$T1 = \text{hash}[1];$

$T2 = \text{hash}[i-1] - \text{hash}[1] * \text{pow}[i-1-1];$

$T3 = \text{hash}[n] - \text{hash}[i] * \text{pow}[n-i];$

$T4 = T2 * \text{pow}[n-i] + T3;$



Friend

这样做看上去是没有问题了，但是在判断重复的时候如果我们只判断前面是否存在前后两个字符串的hash值相等是不准确的，比如AAAAA，无论每次哪一个位置都可以成立，但是实际上原字符串都是AA，所以我们还需要判断一下每次成功的字符串是否是同一个，可以用hash，也可以用hash，如果用hash就必须取余，用map比较慢



Hash算法

当然，hash不仅仅用于字符串，他还可以用于数组。比如：

在二维平面直角坐标系中给定 n 个坐标轴上的点 (x, y) ，判断该点之前是否出现过， $x, y \leq 100000$ ， $n \leq 100000$ 。

Hash算法

这道题最麻烦的地方在于，坐标范围太大，没有办法直接开二维数组，当然可以直接排序，这里介绍一种新的办法——hash。

如果我们直接开数组，需要开100亿数组，但是实际上点就只有10w个，里面的很多空间都是浪费了的，所以我们可以考虑hash，即把每个点的横纵坐标hash一下变成一个值。后面的处理办法就和前面处理字符串一摸一样了。比如

$$Z = (x * 131 + y * 10007) \% 100000007$$

STL——Map

这里讲的map并不是hash的stl，而是基于平衡树的，现在C++11里的map也有以前的平衡树变成了更高效hash表，不过遗憾的是目前NOIP不支持C++11。所以如果用map效率就是 $n \log n$ ，而且特别慢，慎用！！！！

map 基本操作

头文件: `#include<map>`

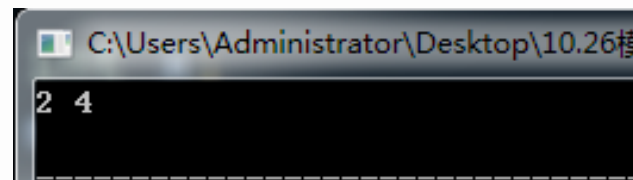
定义: `map<int, int> m;`

插入: `m[i]=x`//键值为i, 实际值为x

插入: `m.insert(make_pair(i, x));`

上面两种插入是有区别的, 第一种是类似数组操作, 相同键值直接覆盖, 第二种插入, 相同键值表示插入失败, 以第一次插入的值为准。

```
map<int,int> m;  
m.insert(make_pair(1,2));  
m.insert(make_pair(1,4));  
m[2]=3;  
m[2]=4;  
printf("%d %d\n",m[1],m[2]);
```



```
C:\Users\Administrator\Desktop\10.26模  
2 4
```

Map 字符串哈希

```
//需要头文件#include<iostream>
map<string,int> m;
for(int i=1;i<=n;i++)
{
    scanf("%s",str);
    if(m[str]==0)
    {
        ans++;
        m[str]=1;
    }
}
printf("%d",ans);
```

点对判重

给定n个坐标轴上的点(x, y), 判断该点在之前是否出现过, $x, y \leq 100000$

```
map<int, map<int, int> > m;  
//注意写法, 数对在后面, 后面两个>用空格隔开  
for(int i=1; i<=n; i++)  
{  
    scanf("%d%d", &x, &y);  
    if(m[x][y]==0)  
    {  
        printf("No\n");  
        m[x][y]=1;  
        //虽然看上去是数组, 但是实际上并不是连续分配空间  
    }  
    else  
        printf("Yes\n");  
}  
printf("%d", ans);  
return 0;
```

三维: `map<int, map<int, map<int, int > > > m;`

二分 + hash

根据前面的hash的学习，感觉hash作用也不是很大，要么效率不高，要么准确度不高，并且有很多可以替代的方法，接下来我们来学习一种用hash骗分的方法——二分+hash。

求解一个字符串中的最长回文子串。

输入：

abcbabcbabcba

abacacbaaab

输出：

13

6

最长回文子串

这是一道manache的模板题，但是很不幸的是manache算是省选算法，暂时不学，但是我们仍然可以用学过的办法来暴力求解，或者说骗分。Manache的时间复杂度为 $O(n)$ ，而我们接下来要学的二分+hash时间复杂度为 $n \log n$ 。

我们先思考一下暴力怎么做，很明显是枚举每一个字符或者中间那个空字符，当做回文串的回文字符，然后再枚举判断是否满足回文的要求。最坏时间复杂度为 $O(n^2)$

最长回文子串

我们来思考一下如何优化，首先，每一个字符都有可能是回文字符的起点，那么枚举是必不可少的，接下来就是比较，也就是判断该位置左右两边的字符串是否相等，暴力比较是 $O(n^2)$ ，那如果我们转换成hash值比较呢？

由于我们不知道长度，貌似再枚举长度，也是 $O(n^2)$ ，我们这里转换一种思想——二分，如果当前长度不可以匹配，但是他的内部一部分可能是可以匹配的，我们没有必要一个长度一个长度的减，而是可以直接二分来枚举。

最长回文子串

比如dcbaaabcf

当我们枚举到第二个a的时候，先枚举长度4
发现dcba和abcf不相等，长度减半为2

发现ba和ab相等，但是此时并不是就完了，
此时我们只是找到了回文串，但是不一定是最长回文串，所以此时回文长度在 $[2, 4)$ 之间，我们再枚举长度3，发现，仍然匹配，此时区间长度为1，就没有必要再枚举了，所以总长度为 $1+3*2=7$.

二分 + hash

1: 和前面用hash做字符串匹配一样，先预处理前缀hash之和。但是值得注意的是注意方向。

比如：abcdcda

算前缀是 $a \rightarrow c$ ，算后缀应该还是 $a \rightarrow c$ ，所以应该分别预处理正向和反向的前缀hash值。

```
zheng[0]=0 ; fan[len+1]=0;
for(int i=1;i<=len;i++)
zheng[i]=zheng[i-1]*131+(str[i]-'a');
for(int i=len;i>=1;i--)
fan[i]=fan[i+1]*131+(str[i]-'a');
```

二分 + hash

2: 枚举可能是回文中心的字符，注意可能是该字符，也可能是该字符后面的那个空位，所以两种情况都需要分别枚举。

```
void Single(int i) //以i为对称中心
{
    int l=0, r=min(i-1, len-i); //枚举回文长度
    while(l<=r)
    {
        int mid=(l+r)/2;
        if(check(i-mid, i-1, i+mid, i+1))
            ans=mid, l=mid+1;
        else
            r=mid-1;
    }
    Max=max(Max, ans*2+1);
}
```

二分 + hash

```
void Double(int i) //以i后面的空位置为对称中心
{
    int l=0, r=min(i-1, len-i-1);
    while(l<=r)
    {
        int mid=(l+r)/2;
        if(check(i-mid, i-1, i+mid+1, i+2))
            ans=mid, l=mid+1;
        else
            r=mid-1;
    }
    Max=max(Max, (ans+1)*2);
}
```

二分 + hash

```
bool check(int l1, int r1, int l2, int r2)
{
    long long tmp1=zheng[r1]-zheng[l1-1]*pow[r1-l1+1];
    long long tmp2=fan[r2]-fan[l2+1]*pow[r1-l1+1];
    return tmp1==tmp2;
}
```



必做练习题

1: 洛谷 P3370 字符串hash (hash和map两种)

2: POJ 3461 Oulipo (hash和kmp两种)

3: 绵实OJ 1431 点对判重

4: POJ 3974 Palindrome

最长双回文

追妹纸

